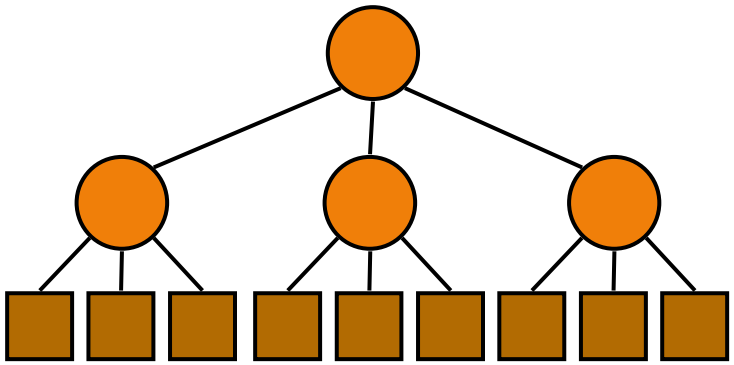
Lecture 07 Divide & Conquer

Learning Outcome

- Divide-and-conquer paradigm (5.2)
- Review Merge-sort (4.1.1)
- Recurrence Equations (5.2.1)
  - Iterative substitution
  - Recursion trees
  - The Master method
- Review Quick-sort (4.3)

Divide-and-Conquer

- Divide-and-conquer is a general algorithm design paradigm:
  - Divide: divide the input data S in two or more disjoint subsets S_1, S_2, …
  - Recur: solve the subproblems recursively
  - Conquer: combine the solutions for S_1, S_2, …, into a solution for S
- The base case for the recursion are subproblems of constant size
- Analysis can be done using **recurrence equations**



Merge-Sort Review

```
Algorithm MergeSort (S, left, right)
  Input: array S, left index, right index.
  Output: array S sorted.

  if left < right
    // Divide: divide S into two nearly equal halves.
    mid = (left + right) / 2
    // Recur: sort each half recursively.
    MergeSort (S, left, mid)      // left half, S1
    MergeSort (S, mid + 1, right) // right half, S2
    // Conquer: merge the two halves into a sorted S.
    Merge (S, left, mid, right)
```

Merge Review

- The conquer step of merge-sort consists of merging two sorted sequences L and R into a sorted sequence A[left]…A[right]
- Merging two sorted sequences, each with $n/2$ elements, takes $O(n)$ time

```
Algorithm Merge (S, left, mid, right)
  Input: array S, left index, mid index,
   right index.
  Output: sorted array S[left]...S[right].

  L = new LinkedList from S[left]...S[mid]
  R = new LinkedList from S[mid+1]...S[right]

  k = left
  while not L.isEmpty() and not R.isEmpty()
    if L.first() < R.first()
      S[k++] = L.removeFirst()
    else
      S[k++] = R.removeFirst()
  while not L.isEmpty()
    S[k++] = L.removeFirst()
  while not R.isEmpty()
    S[k++] = R.removeFirst()
```

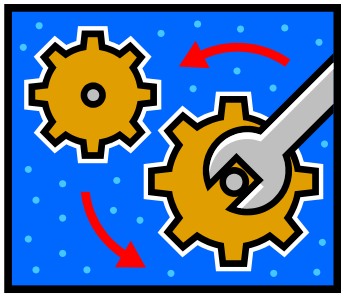
Recurrence Equation Analysis

- The conquer step of merge-sort consists of merging two sorted sequences, each with $n/2$ elements and implemented by means of a doubly linked list, takes at most bn steps, for some constant b.
- Likewise, the basis case ($n < 2$) will take at b most steps.
- Therefore, if we let $T(n)$ denote the running time of merge-sort:

$$T(n) = \begin{cases} b & \text{if } n < 2 \\ 2T(n/2) + bn & \text{if } n \geq 2 \end{cases}$$

- We can therefore analyze the running time of merge-sort by finding a **closed form solution** to the above equation.
  - That is, a solution that has $T(n)$ only on the left-hand side.

Iterative Substitution



- In the iterative substitution, or "plug-and-chug," technique, we iteratively apply the recurrence equation to itself and see if we can find a pattern:

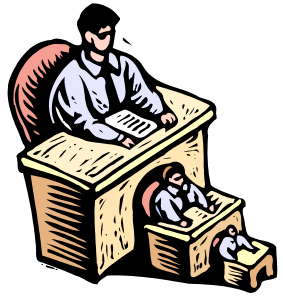
$$\begin{aligned} T(n) &= 2T(n/2) + bn \\ &= 2(2T(n/2^2) + b(n/2)) + bn \\ &= 2^2 T(n/2^2) + 2bn \\ &= 2^3 T(n/2^3) + 3bn \\ &= 2^4 T(n/2^4) + 4bn \\ &= \ldots \\ &= 2^i T(n/2^i) + ibn \end{aligned}$$

- Note that base case, T(1)=b, occurs when $2^i=n$. That is, i = log n.
- So,

$$T(n) = bn + bn \log n$$

- Thus, T(n) is O(n log n).

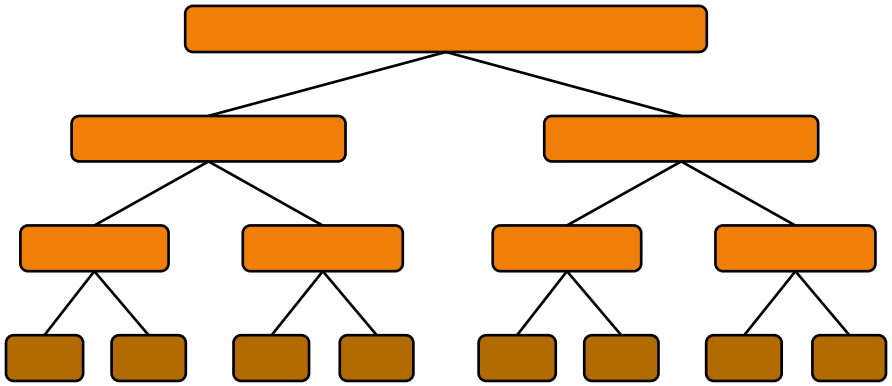
The Recursion Tree



- Draw the recursion tree for the recurrence relation and look for a pattern:

$$T(n) = \begin{cases} b & \text{if } n < 2 \\ 2T(n/2) + bn & \text{if } n \geq 2 \end{cases}$$

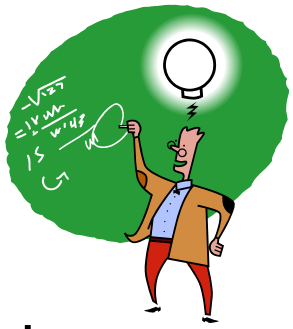
depth	T's	size	time
0	1	n	bn
1	2	$n/2$	bn
i	2^i	$n/2^i$	bn
…	…	…	…



Total time = $bn + bn \log n$

(last level + all previous levels)

Master Method



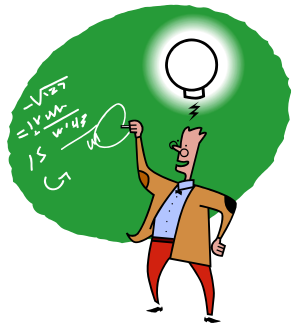
- Many divide-and-conquer recurrence equations have the form:

$$T(n) = \begin{cases} c & \text{if } n < d \\ aT(n/b) + f(n) & \text{if } n \geq d \end{cases}$$

- The Master Theorem:

1. if $f(n)$ is $O(n^{\log_b a - \varepsilon})$, then $T(n)$ is $\Theta(n^{\log_b a})$
2. if $f(n)$ is $\Theta(n^{\log_b a} \log^k n)$, then $T(n)$ is $\Theta(n^{\log_b a} \log^{k+1} n)$
3. if $f(n)$ is $\Omega(n^{\log_b a + \varepsilon})$, then $T(n)$ is $\Theta(f(n))$, provided $af(n/b) \leq \delta f(n)$ for some $\delta < 1$.

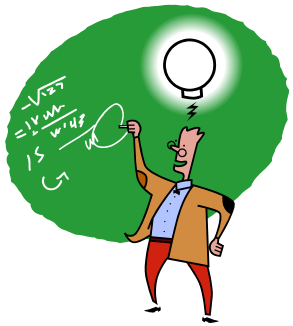
Master Method, Example 1



- The form: $$T(n) = \begin{cases} c & \text{if } n < d \\ aT(n/b) + f(n) & \text{if } n \geq d \end{cases}$$
- The Master Theorem:
  1. if $f(n)$ is $O(n^{\log_b a - \varepsilon})$, then $T(n)$ is $\Theta(n^{\log_b a})$
  2. if $f(n)$ is $\Theta(n^{\log_b a} \log^k n)$, then $T(n)$ is $\Theta(n^{\log_b a} \log^{k+1} n)$
  3. if $f(n)$ is $\Omega(n^{\log_b a + \varepsilon})$, then $T(n)$ is $\Theta(f(n))$, provided $af(n/b) \leq \delta f(n)$ for some $\delta < 1$.
- Example: $$T(n) = 4T(n/2) + n$$

Solution: $\log_b a = 2$
Case 1: $n < n^2$, true
$T(n)$ is $\Theta(n^2)$

Master Method, Example 2



- The form: $$T(n) = \begin{cases} c & \text{if } n < d \\ aT(n/b) + f(n) & \text{if } n \geq d \end{cases}$$
- The Master Theorem:
  1. if $f(n)$ is $O(n^{\log_b a - \varepsilon})$, then $T(n)$ is $\Theta(n^{\log_b a})$
  2. if $f(n)$ is $\Theta(n^{\log_b a} \log^k n)$, then $T(n)$ is $\Theta(n^{\log_b a} \log^{k+1} n)$
  3. if $f(n)$ is $\Omega(n^{\log_b a + \varepsilon})$, then $T(n)$ is $\Theta(f(n))$, provided $af(n/b) \leq \delta f(n)$ for some $\delta < 1$.
- Example:

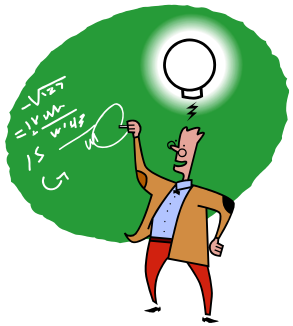
$$T(n) = 2T(n/2) + n \log n$$

Solution: $\log_b a = 1$

Case 2: $n \log n = n^1 \log^k n$, true if $k=1$

$T(n)$ is $\Theta(n \log^2 n)$

Master Method, Example 3



- The form: $$T(n) = \begin{cases} c & \text{if } n < d \\ aT(n/b) + f(n) & \text{if } n \geq d \end{cases}$$
- The Master Theorem:
  1. if $f(n)$ is $O(n^{\log_b a - \varepsilon})$, then $T(n)$ is $\Theta(n^{\log_b a})$
  2. if $f(n)$ is $\Theta(n^{\log_b a} \log^k n)$, then $T(n)$ is $\Theta(n^{\log_b a} \log^{k+1} n)$
  3. if $f(n)$ is $\Omega(n^{\log_b a + \varepsilon})$, then $T(n)$ is $\Theta(f(n))$, provided $af(n/b) \leq \delta f(n)$ for some $\delta < 1$.
- Example:

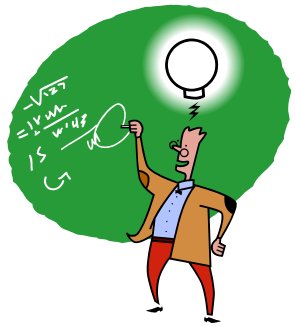
$$T(n) = T(n/3) + n \log n$$

Solution: $\log_b a = 0$

Case 3: $n \log n > n^0$ true

$T(n)$ is $\Theta(n \log n)$

Master Method, Example 4



- The form: $T(n) = \begin{cases} c & \text{if } n < d \\ aT(n/b) + f(n) & \text{if } n \geq d \end{cases}$
- The Master Theorem:
  1. if $f(n)$ is $O(n^{\log_b a - \varepsilon})$, then $T(n)$ is $\Theta(n^{\log_b a})$
  2. if $f(n)$ is $\Theta(n^{\log_b a} \log^k n)$, then $T(n)$ is $\Theta(n^{\log_b a} \log^{k+1} n)$
  3. if $f(n)$ is $\Omega(n^{\log_b a + \varepsilon})$, then $T(n)$ is $\Theta(f(n))$, provided $af(n/b) \leq \delta f(n)$ for some $\delta < 1$.
- Example:

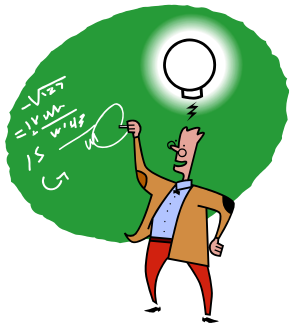
$$T(n) = 8T(n/2) + n^2$$

Solution: $\log_b a = 3$

Case 1: $n^2 < n^3$, true

T(n) is $\Theta(n^3)$

Master Method, Example 5



- The form: $$T(n) = \begin{cases} c & \text{if } n < d \\ aT(n/b) + f(n) & \text{if } n \geq d \end{cases}$$
- The Master Theorem:
  1. if $f(n)$ is $O(n^{\log_b a - \varepsilon})$, then $T(n)$ is $\Theta(n^{\log_b a})$
  2. if $f(n)$ is $\Theta(n^{\log_b a} \log^k n)$, then $T(n)$ is $\Theta(n^{\log_b a} \log^{k+1} n)$
  3. if $f(n)$ is $\Omega(n^{\log_b a + \varepsilon})$, then $T(n)$ is $\Theta(f(n))$, provided $af(n/b) \leq \delta f(n)$ for some $\delta < 1$.
- Example:

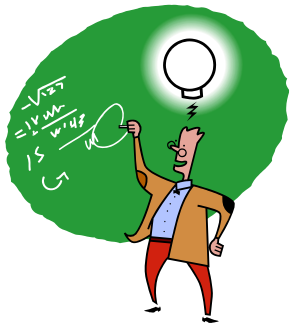
$$T(n) = 9T(n/3) + n^3$$

Solution: $\log_b a = 2$

Case 3: $n^3 > n^2$, true

T(n) is $\Theta(n^3)$

Master Method, Example 6



- The form: $$T(n) = \begin{cases} c & \text{if } n < d \\ aT(n/b) + f(n) & \text{if } n \geq d \end{cases}$$
- The Master Theorem:
  1. if $f(n)$ is $O(n^{\log_b a - \varepsilon})$, then $T(n)$ is $\Theta(n^{\log_b a})$
  2. if $f(n)$ is $\Theta(n^{\log_b a} \log^k n)$, then $T(n)$ is $\Theta(n^{\log_b a} \log^{k+1} n)$
  3. if $f(n)$ is $\Omega(n^{\log_b a + \varepsilon})$, then $T(n)$ is $\Theta(f(n))$, provided $af(n/b) \leq \delta f(n)$ for some $\delta < 1$.
- Example:

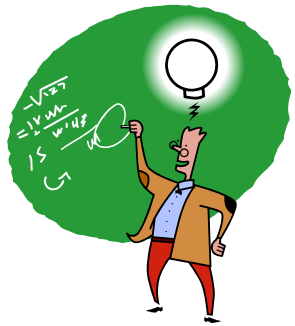
$$T(n) = T(n/2) + 1 \quad \text{(binary search)}$$

Solution: $\log_b a = 0$

Case 2: $1 = n^0 \log^k n$, true if $k = 0$

$T(n)$ is $\Theta(\log n)$

Master Method, Example 7



- The form: $$T(n) = \begin{cases} c & \text{if } n < d \\ aT(n/b) + f(n) & \text{if } n \geq d \end{cases}$$
- The Master Theorem:
  1. if $f(n)$ is $O(n^{\log_b a - \varepsilon})$, then $T(n)$ is $\Theta(n^{\log_b a})$
  2. if $f(n)$ is $\Theta(n^{\log_b a} \log^k n)$, then $T(n)$ is $\Theta(n^{\log_b a} \log^{k+1} n)$
  3. if $f(n)$ is $\Omega(n^{\log_b a + \varepsilon})$, then $T(n)$ is $\Theta(f(n))$, provided $af(n/b) \leq \delta f(n)$ for some $\delta < 1$.
- Example:

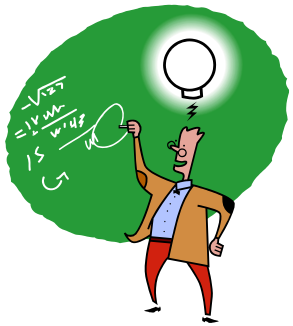
$$T(n) = 2T(n/2) + \log n$$ (heap construction)

Solution: $\log_b a = 1$

Case 1: $\log n < n^1$, true

$T(n)$ is $\Theta(n)$

Master Method, Example 8

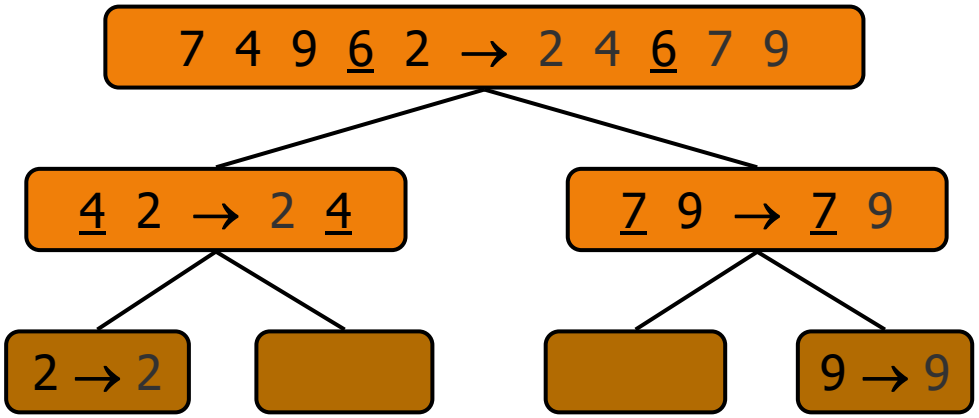


- The form: $$T(n) = \begin{cases} c & \text{if } n < d \\ aT(n/b) + f(n) & \text{if } n \geq d \end{cases}$$
- The Master Theorem:
  1. if $f(n)$ is $O(n^{\log_b a - \varepsilon})$, then $T(n)$ is $\Theta(n^{\log_b a})$
  2. if $f(n)$ is $\Theta(n^{\log_b a} \log^k n)$, then $T(n)$ is $\Theta(n^{\log_b a} \log^{k+1} n)$
  3. if $f(n)$ is $\Omega(n^{\log_b a + \varepsilon})$, then $T(n)$ is $\Theta(f(n))$, provided $af(n/b) \leq \delta f(n)$ for some $\delta < 1$.
- Example:

$$T(n) = 2T(n-2) + \log n$$

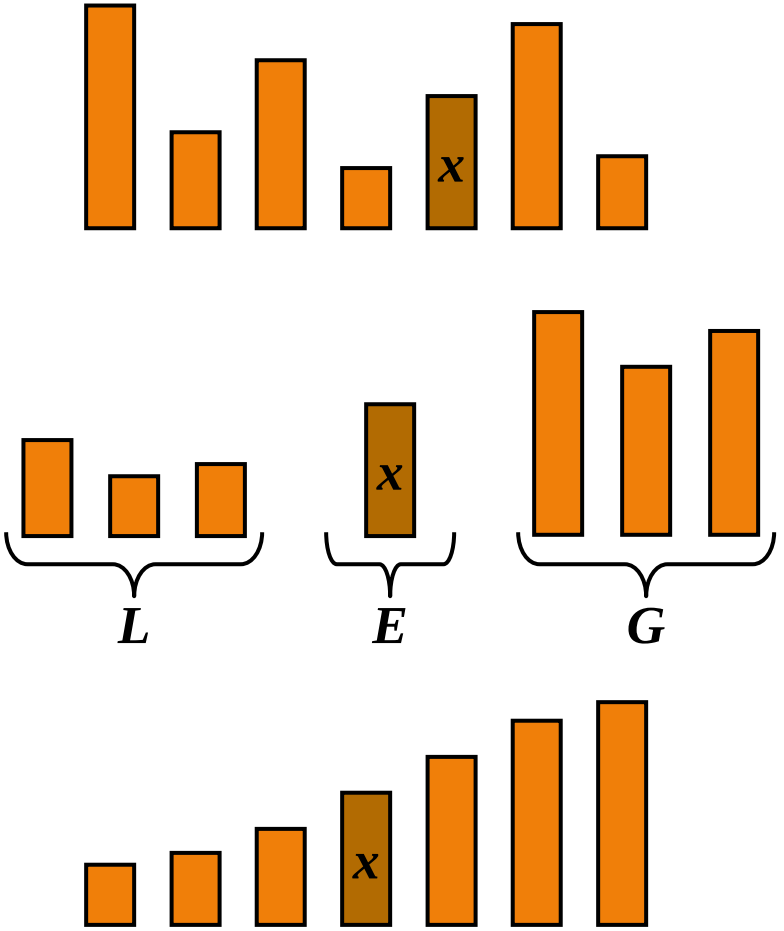
Cannot be solved by Master Method because T(n) is not in the form of aT(n/b)+f(n).

Quick-Sort



Quick-Sort

- Quick-sort is a sorting algorithm based on the divide-and-conquer paradigm:
  - Divide: pick a random element x (called pivot) and partition S into
    - L elements less than x
    - E elements equal x
    - G elements greater than x
  - Recur: recursively sort L and G
  - Conquer: join L, E and G
  - Recall that Lec05a picks the last element as pivot



Algorithm QuickSort

```
Algorithm QuickSort (S, left, right)
  Input: Array S, left index, right index.
  Output: S is sorted.
  if left < right
    // pivot index
    pi = partition (S, left, right)
    QuickSort (S, left, pi-1)   // L, E
    QuickSort (S, pi+1, right)  // G
```

Partition

- The partition step of quick-sort takes $O(n)$ time.

```
Algorithm partition(S)
Input: Array S.
Output: All elements < pivot are placed
before pivot, and all elements > pivot
are placed after pivot.

  L, E, G = empty sequences
  p = S.last() // pivot
  while not S.isEmpty()
    e = S.removeFirst()
    if e < p
      L.insertLast(e)
    else if e = p
      E.insertLast(e)
    else // e > p
      G.insertLast(e)

  while not L.isEmpty()
    S.insertLast(L.removeFirst())
  pi = S.size() // pivot index
  while not E.isEmpty()
    S.insertLast(E.removeFirst())
  while not G.isEmpty()
    S.insertLast(G.removeFirst())

  return pi
```

Quick-Sort Worst-case

- First element as pivot (7 partition calls):

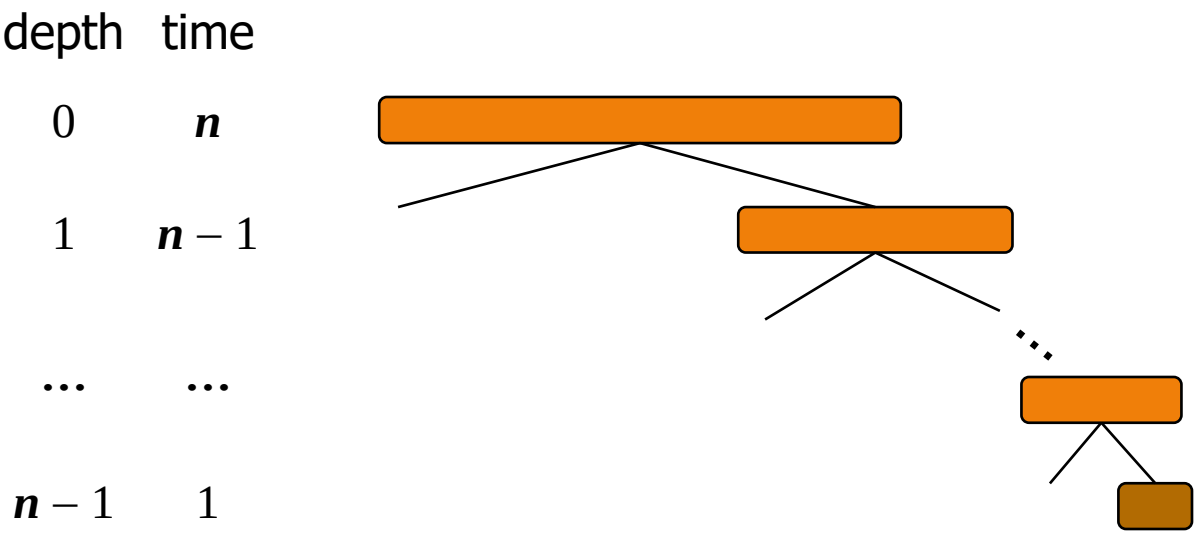
1. 1 2 3 4 5 6 7 8
2. 1* 2 3 4 5 6 7 8
3. 1* 2* 3 4 5 6 7 8
4. 1* 2* 3* 4 5 6 7 8
5. 1* 2* 3* 4* 5 6 7 8
6. 1* 2* 3* 4* 5* 6 7 8
7. 1* 2* 3* 4* 5* 6* 7 8
8. 1* 2* 3* 4* 5* 6* 7* 8

Worst-case Running Time

- The worst case for quick-sort occurs when the pivot is the unique minimum or maximum element (already sorted)
- One of L and G has size $n - 1$ and the other has size 0
- The running time is proportional to the sum

$$n + (n - 1) + \ldots + 2 + 1$$

- Thus, the worst-case running time of quick-sort is $O(n^2)$



Random Pivot

- Random pivot can reduce quick-sort worst case.
- Sample run (4 partition calls):

1. 1 2 3 4 5 6 7 8
2. 1 2 3 4 5* 6 7 8
3. 1 2* 3 4 5* 6 7 8
4. 1 2* 3* 4 5* 6 7 8
5. 1 2* 3* 4 5* 6 7* 8

Expected Running Time

- Quick-sort performs the best when L and G are of equal size. In this case, a single quick-sort call involves bn work of partition plus two recursive calls on lists of size n/2, hence the recurrence relation is (same as merge-sort)

  $$T(n) = 2T(n/2) + bn$$

- From the Master Theorem, the best-case running time of quick-sort is O(n log n).
- For average case, a random pivot reduces the probability of worst case, the expected running time is also O(n log n).